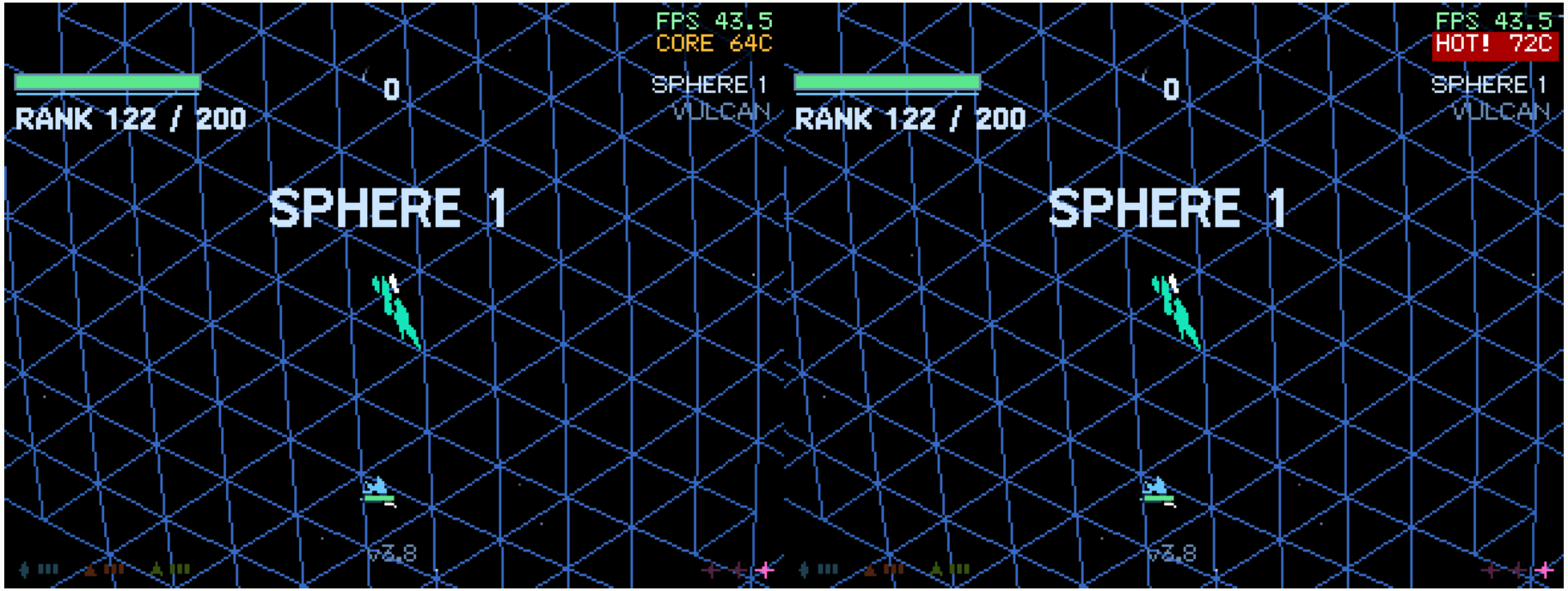
PICO 2 GAME CONSOLE

Devour Sphere on RP2350

Hardware and software choices, round by round



RP2350 A2 · 264 MHz · ST7789 320×240 SPI · 2 cores · 42-45 FPS · 38 ms input-to-screen

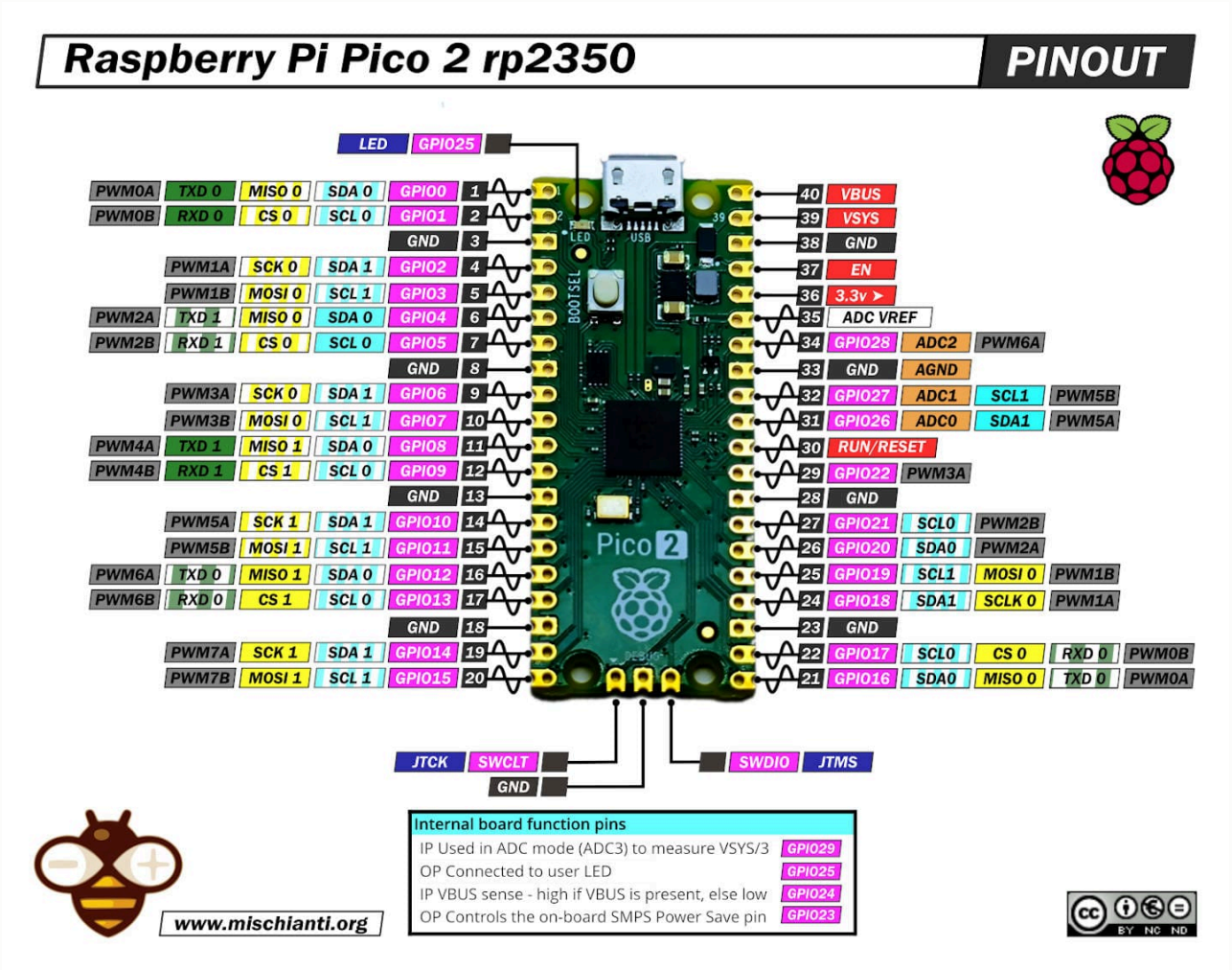
BUILD-UP

Seventeen rounds to the current revision

R1	Speed: code in RAM, clock, SPI rate, sim on core 1, late launch
R2	Epic original music, textured world surface
R3	Player skinned with the world, HUD legend
R4	World select screen and cut scene
R5	Button names, per-weapon high scores, stronger upgrades
R6	Stored music: ADPCM from flash through a PWM DAC
R7	Sleep mode on B, resume at the same frame
R8-9	264 MHz @ 1.15 V, screen-only sleep with cores asleep
R10	Datasheet errata fixes, core temperature indicator
R11	Work-sharing scheduler across both cores
R12	Colour-table animation, whole and louder music
R13	Height map + ground animation on core 1, yield + WFE: no FPS cost
R14-15	Pin probe tried and reverted; Makefile pins the shipping build
R16	New board: ST7789 panel, battery current measured, synthwave music
R17	Bodies lit by the world's sun (ShapoGFX lights)

HARDWARE

The console



- Pico 2, RP2350 **A2**, QFN-60
- ST7789 320×240 on SPI0: GP16-21 (R16)
- Analog stick: GP26/27 = ADC0/1
- Two buttons: GP8 = A, GP9 = B
- Speaker: GP15 PWM → NPN → 8 Ω
- Power: 3× AA into VSYS
- All pins in one file: *board_pins.h*

DEBUG DETAIL

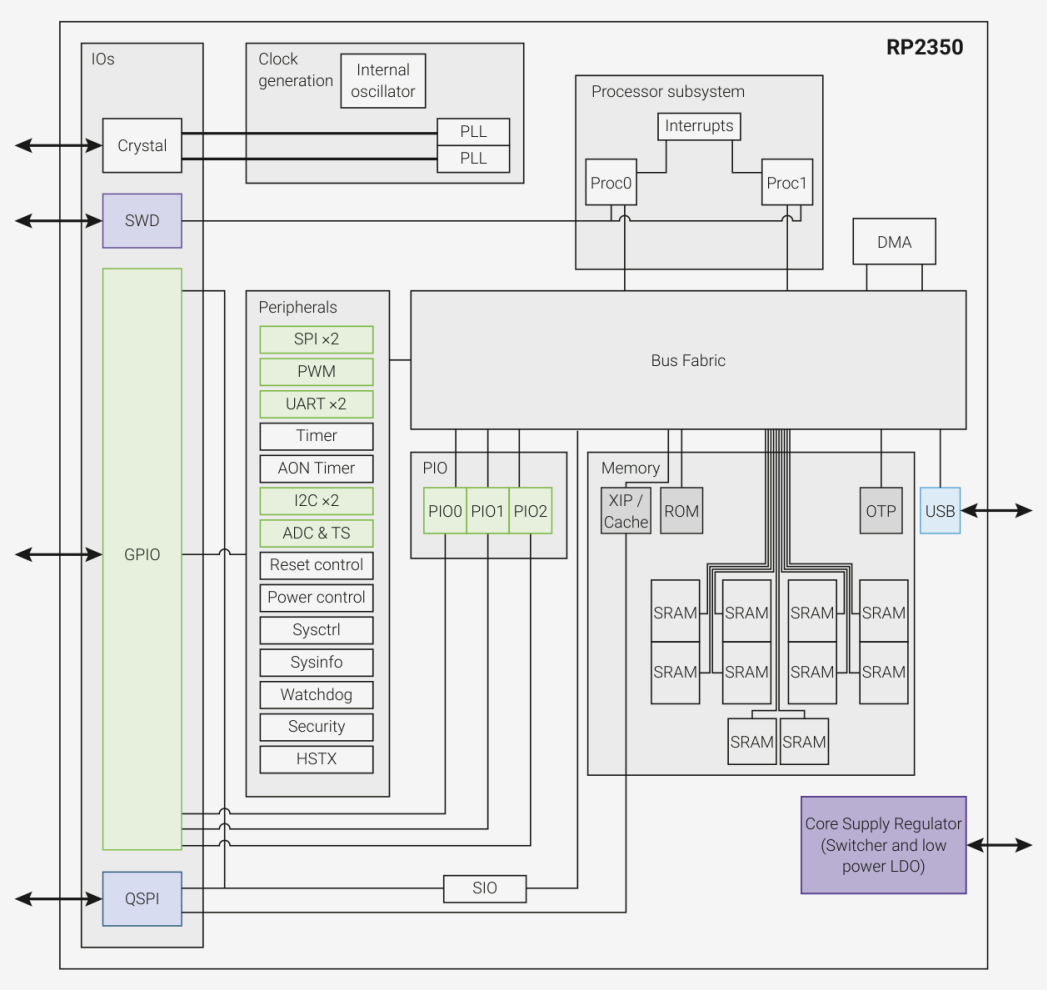
The once-a-second log prints the button mask (**btn 080**). On the first board it showed GP10 floats and GP11 reads pressed for good, so game logic uses only GP8 and GP9.

Pico 2 pinout (board.png)

HARDWARE

What we use on the chip

Figure 1.
A system overview of the RP2350 chip

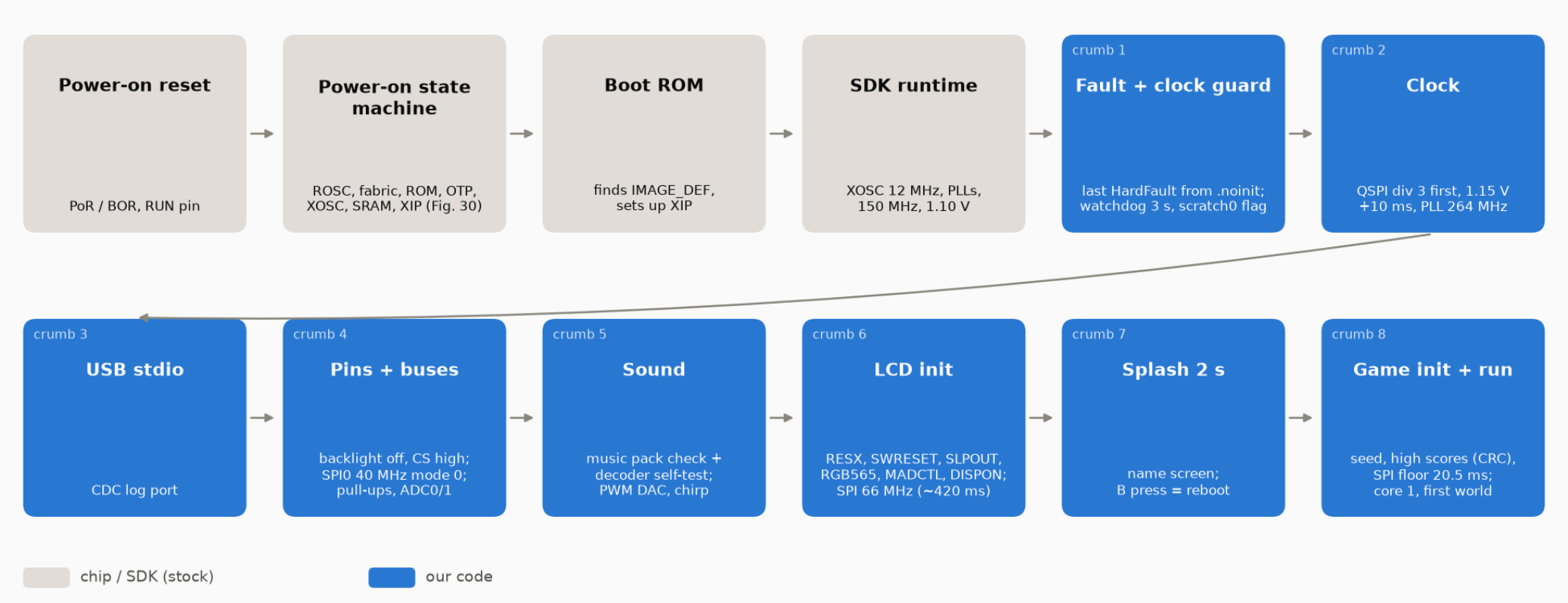


- Two Cortex-M33 cores: render | simulate
- SPI0 + DMA: display bands
- PWM slice: 31.25 kHz audio DAC
- ADC: stick + die temperature
- QSPI XIP: firmware, music, scores
- Watchdog + scratch: clock guard, crumbs

RP2350 datasheet, Fig. 1, system overview

HARDWARE + SOFTWARE

Power-up to the first frame



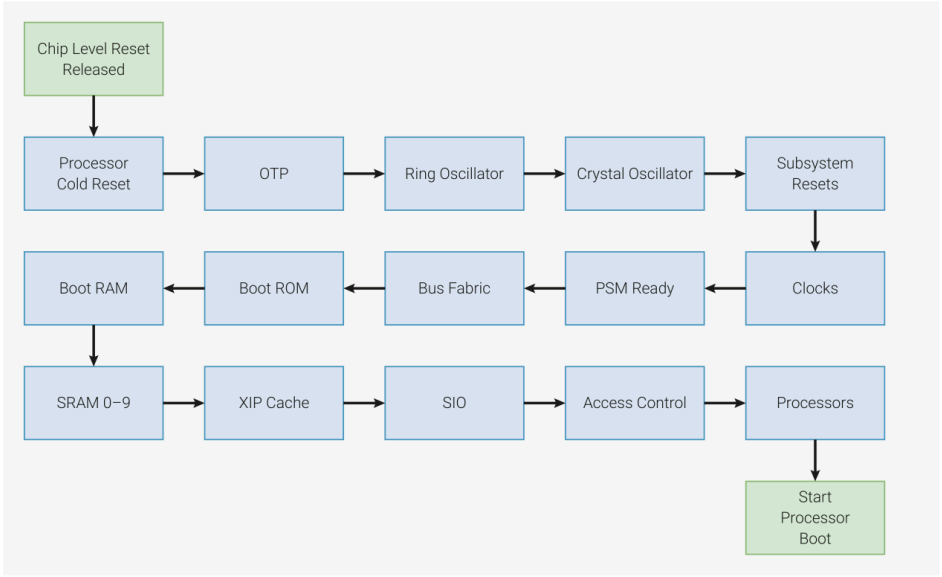
DEBUG DETAIL

Each step writes a breadcrumb to watchdog **scratch1**; **scratch0** holds the clock-guard flag. Both survive the watchdog's system reset (datasheet §7.2, §12.9.5), so **prevcrumb** in the CFG log says where a boot stopped. The bootrom owns scratch4-7 (watchdog boot, §5.2.4): we never touch them.

HARDWARE + SOFTWARE

Our boot steps, and the rule behind each

Figure 30.
Power-on State Machine Sequence

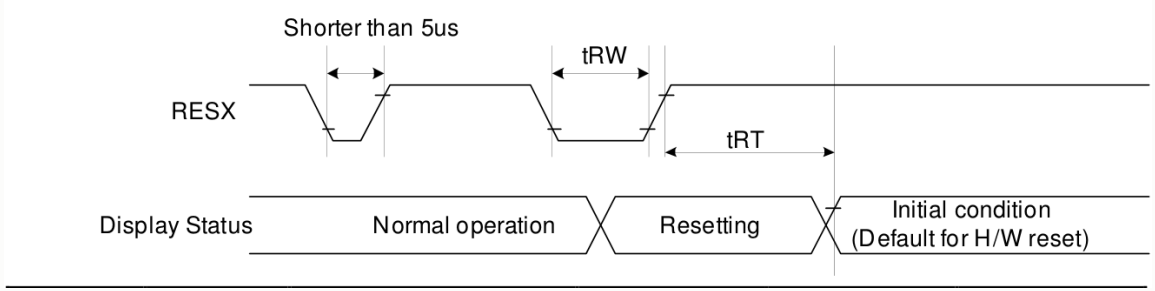


RP2350 datasheet, Fig. 30, power-on state machine

STEP	WHAT	WHY
Guard	read watchdog REASON + scratch, arm 3 s watchdog	hang → 150 MHz fallback
Flash	QSPI divider 3 before the PLL moves	flash ≤ 100 MHz (88 MHz)
Voltage	1.15 V, 10 ms settle, then PLL	voltage up before clock up
PLL	264 MHz from 12 MHz XOSC, clk_peri = clk_sys	SPI 66 MHz = /4
Pins	backlight off, CS high, RST high first	no garbage on the panel
Inputs	pull-ups on buttons, ADC on GP26/27	pressed = low; ADC pads IE off
Audio	pack header + 1 s decode self-test	synth fallback if not bit-exact
Chirp	short tone at power-on	speaker wiring check
SPI floor	time one full screen, nothing else running	20.5 ms, logged as xfer
Core 1	launch, flash_safe_execute_core_init	safe high-score writes
Proven	5 s of frames clears scratch0	later resets keep 264 MHz

HARDWARE (ILI9341, BOARD UNTIL R15)

Panel power-up and sleep timing



ILI9341 datasheet V1.11, §15.4 reset timing

- Backlight off while asleep; GRAM keeps the frame
- Init at 40 MHz SPI, then 66 MHz for pixels

STEP	OURS	DATASHEET MIN
RESX low	20 ms	10 µs (tRW)
After RESX	120 ms	5 ms, 120 ms before SLPOUT
SWRESET	120 ms	5 ms (120 in Sleep Out)
SLPOUT	120 ms	5 ms, 120 before SLPIN
COLMOD	0x55	16 bit/pixel
DISPON	20 ms	-
Sleep	DISPOFF, 5 ms, SLPIN, 6 ms	5 ms after SLPIN
Wake	SLPOUT, 120 ms, DISPON	120 ms after SLPIN

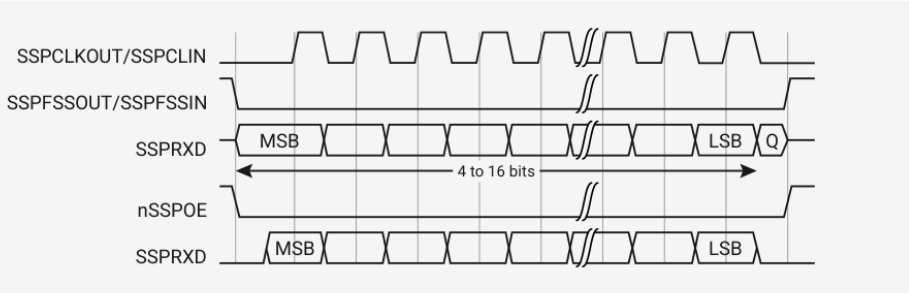
DEBUG DETAIL

Every delay meets the ILI9341 rules (§8.2.2 SWRESET, §8.2.11 SLPIN, §8.2.12 SLPOUT, §15.4 reset). The wake shows the old frame at once: the panel's GRAM is kept through SLPIN.

HARDWARE + SOFTWARE

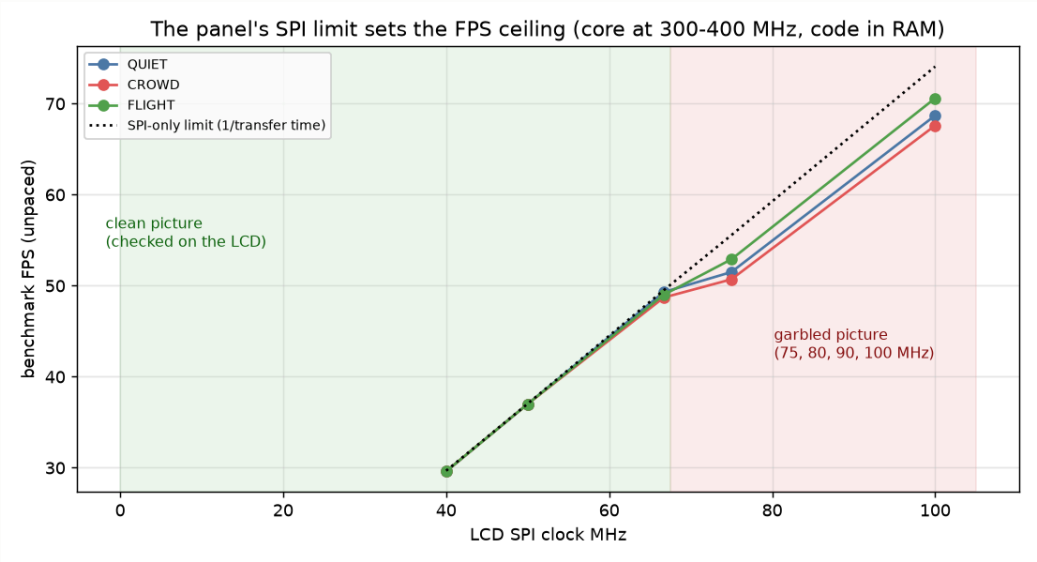
Display link: the frame-rate ceiling

Figure 98.
Motorola SPI frame format, single transfer, with SPO=0 and SPH=0



RP2350 datasheet, Fig. 98, SPI mode 0 (SPO=0, SPH=0)

- RGB565, byte-swapped buffers
- 40-row bands, DMA double buffer
- 66 MHz = 264 MHz / 4
- Full frame 20.5 ms → max 48.8 FPS

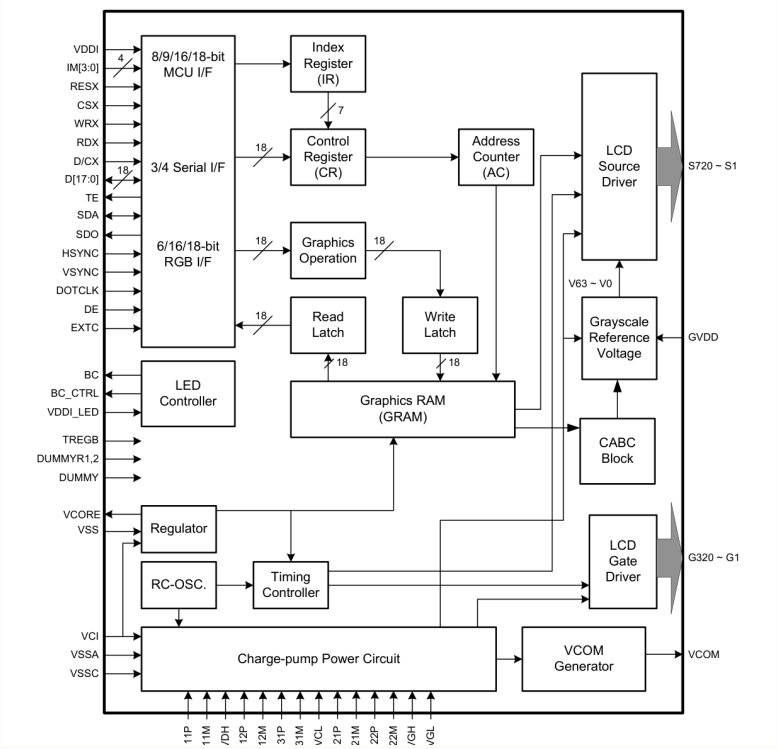


SPI rate sweep (docs/perf/04)

DEBUG DETAIL

The log cannot see a garbled picture. *make spitest* draws a test pattern; the panel was checked by eye: clean up to 66.7 MHz. That set the SPI limit.

ILI9341: what the panel allowed



ILI9341 datasheet V1.11, §3 block diagram

Signal	Symbol	Parameter	min	max	Unit	Description
CSX	tcss	Chip select time (Write)	40	-	ns	
	tcsh	Chip select hold time (Read)	40	-	ns	
SCL	twc	Serial clock cycle (Write)	100	-	ns	
	twrh	SCL “H” pulse width (Write)	40	-	ns	
	twrl	SCL “L” pulse width (Write)	40	-	ns	
	trc	Serial clock cycle (Read)	150	-	ns	
	trdh	SCL “H” pulse width (Read)	60	-	ns	
	trdl	SCL “L” pulse width (Read)	60	-	ns	
D/CX	tas	D/CX setup time	10	-		
	tah	D/CX hold time (Write / Read)	10	-		
SDA / SDI (Input)	tds	Data setup time (Write)	30	-	ns	
	tdh	Data hold time (Write)	30	-	ns	
SDA / SDO (Output)	tacc	Access time (Read)	10	-	ns	For maximum CL=30pF
	tod	Output disable time (Read)	10	50	ns	For minimum CL=8pF

Note: Ta = 25 ℃, VDDI=1.65V to 3.3V, VCI=2.5V to 3.3V, AGND=VSS=0V

ILI9341 datasheet V1.11, §18.3.4, 4-line SPI timing

Write clock spec ≥ 100 ns (10 MHz); run at 15 ns (66 MHz), clean by eye

Link only 4-line SPI: 153.6 KB per frame → 20.5 ms → 48.8 FPS max

GRAM 172,800 B on the panel: bands are enough; image kept in sleep

No TE pin not wired: no sync to the ~70 Hz refresh (FRMCTR1 default)

No MISO write-only panel, nothing read back

DEBUG DETAIL

6.6× the rated clock works on this panel; 75 MHz garbles. *make spitest* decides.

HARDWARE + SOFTWARE

New board: ST7789 panel (Round 16)

SETTING	ILI9341	ST7789	WHY
MADCTL	0xE8	0xA0	MX off: vertical flip; BGR off
INVON	no	no	tried: inverted colours
DMA byte swap	on	on	tried off: noise
Joystick U/D	as wired	flipped	JOY_INVERT_UD

- Same commands, same 66 MHz SPI
- Three settings, all in *SHIP_ARGS*
- Root cause of the colours: RGB vs BGR order

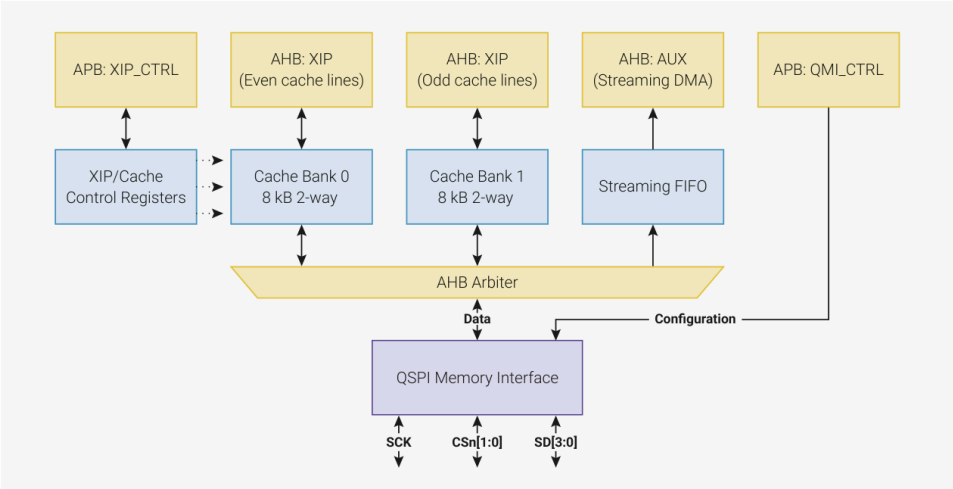
DEBUG DETAIL

Symptom: blue sea orange, Mars bluish, green and black/white unchanged. Only red and blue had traded places, which is MADCTL bit 3 (BGR), not inversion or byte order. Two wrong guesses first (INVON, DMA swap off) were each flashed and judged on the LCD by the user, and reverted.

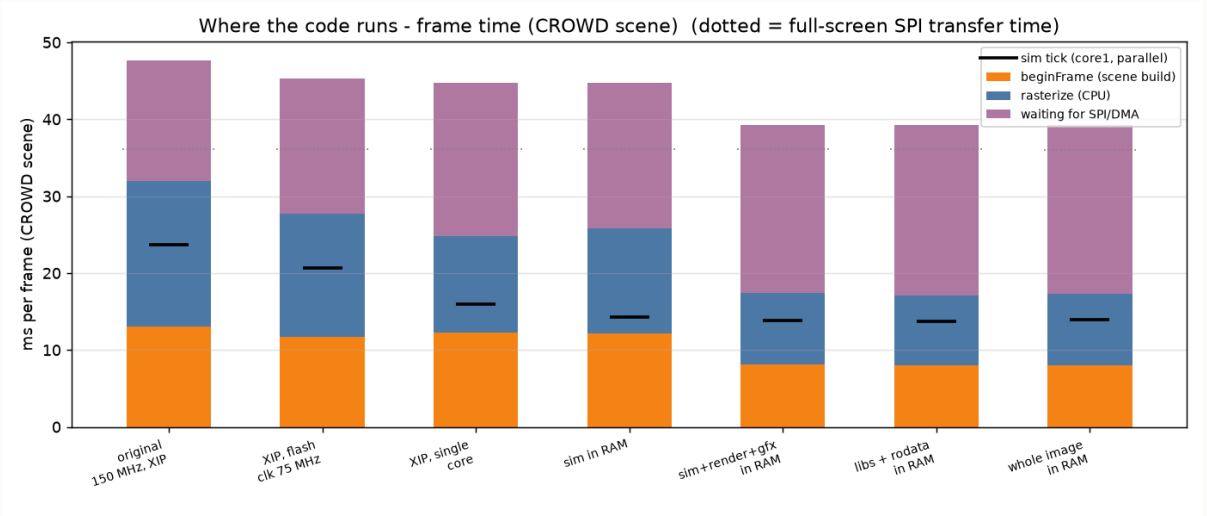
SOFTWARE

Hot code out of flash

Figure 16.
Flash execute-in-place (XIP) subsystem. The cache is split into two banks for performance, but behaves as a single 16 kB cache. XIP accesses first query the cache. If a cache entry is not found, the QMI generates an external serial access, adds the resulting data to the cache, and forwards it on to the system bus (for reads) or merges it with the AHB write data (for writes).



RP2350 datasheet, Fig. 16, XIP subsystem (16 KB cache)



Frame time by code placement (docs/perf/02)

- sim + render + gfx run from SRAM
- Sim tick 23.7 → 13.9 ms (150 MHz)
- Raster 18.9 → 9.3 ms

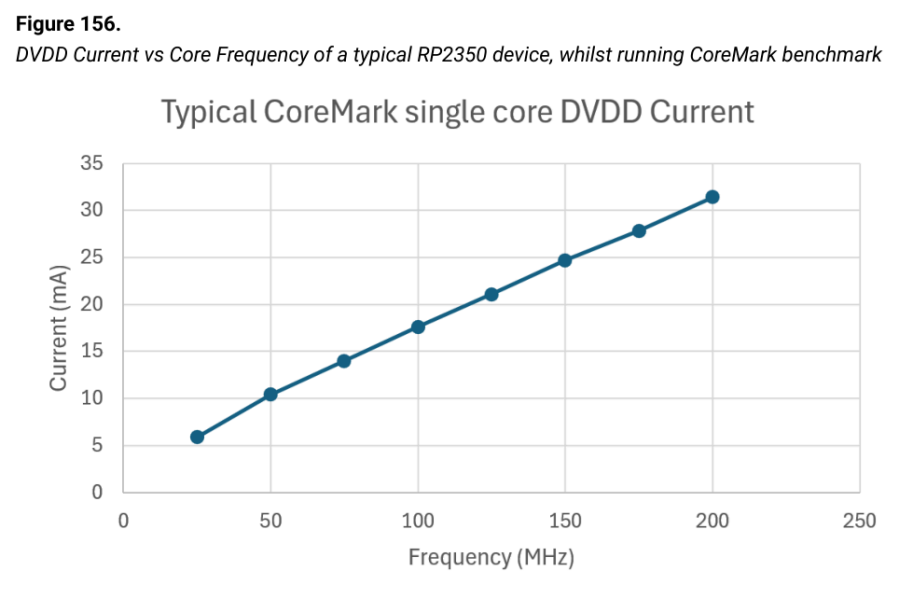
DEBUG DETAIL

Both cores shared one 16 KB XIP cache at a 50 MHz flash clock. Moving three libraries to RAM (*GAME_RAM_CODE*) matched a full copy-to-RAM and left RAM for textures.

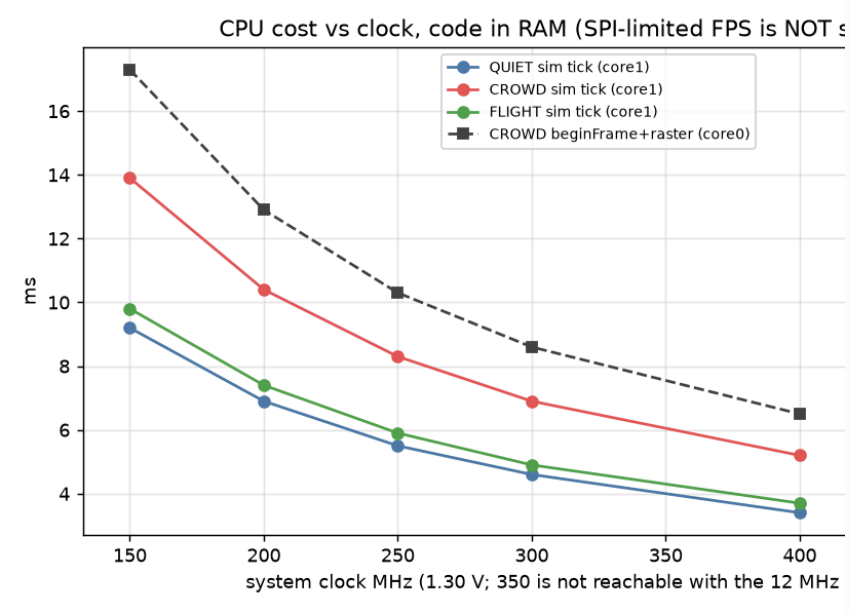
HARDWARE

Clock and core voltage: 264 MHz @ 1.15 V

Figure 156.
DVDD Current vs Core Frequency of a typical RP2350 device, whilst running CoreMark benchmark



RP2350 datasheet, Fig. 156, DVDD current vs core frequency



CPU cost vs clock, 150-400 MHz (docs/perf/03)

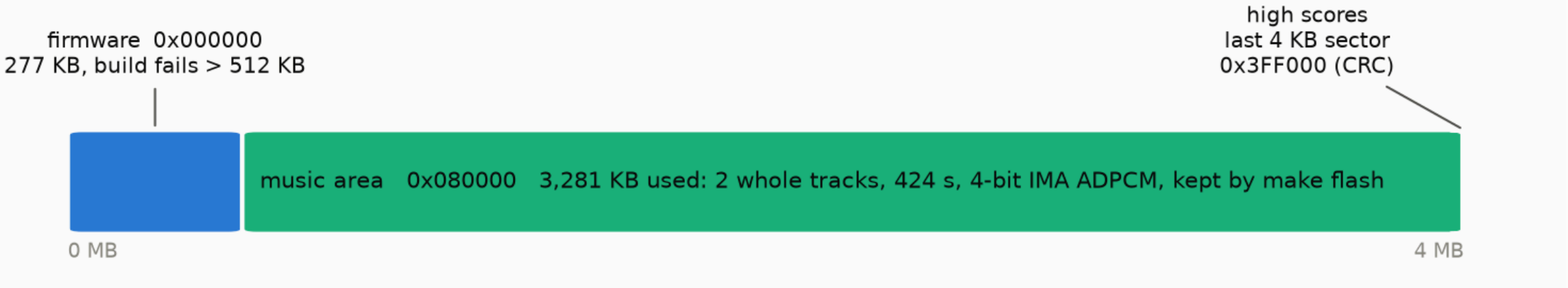
- 264 MHz gives SPI divider 4 = 66 MHz
- 1.15 V; flash clock kept ≤ 100 MHz
- Current rises with clock: no extra MHz

DEBUG DETAIL

Clock guard: a 3 s watchdog over the first 5 s of frames; a hang reboots into 150 MHz (**fallback 1**). Breadcrumbs in watchdog scratch show how far the last boot got (**prevcrumb**).

SOFTWARE

Flash map and safe writes



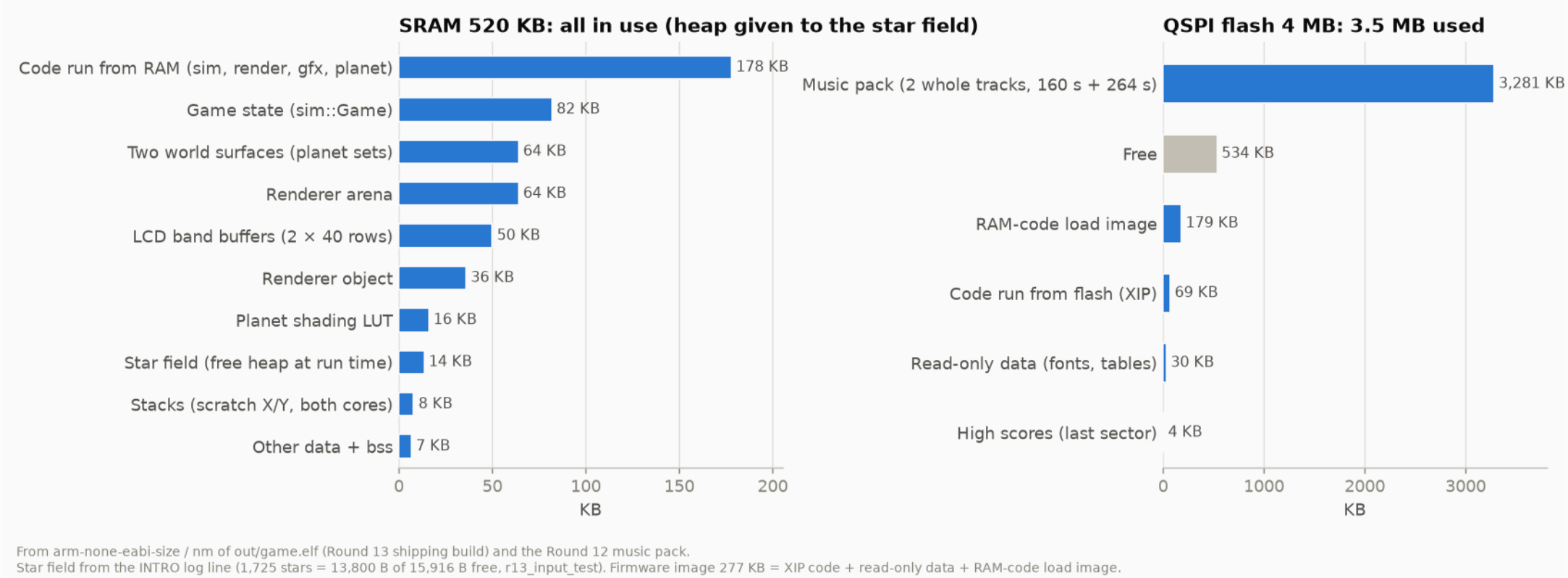
- High scores: CRC'd record, 3 weapons
- Written only off-play (title, game over)
- *flash_safe_execute* pauses core 1
- Music pack survives *make flash*

DEBUG DETAIL

The ROM flash routines reset the QSPI divider to a 150 MHz value. *high_score.cpp* restores it, and the code between ROM call and restore runs from RAM (*__not_in_flash_func*). Checked by the scripted game-over test (**HS save**).

SOFTWARE

RAM and flash footprint



- No framebuffer: 150 KB won't fit; 2 × 25 KB bands
- 178 KB of hot code in SRAM
- Star field takes the free heap, keeps 2 KB

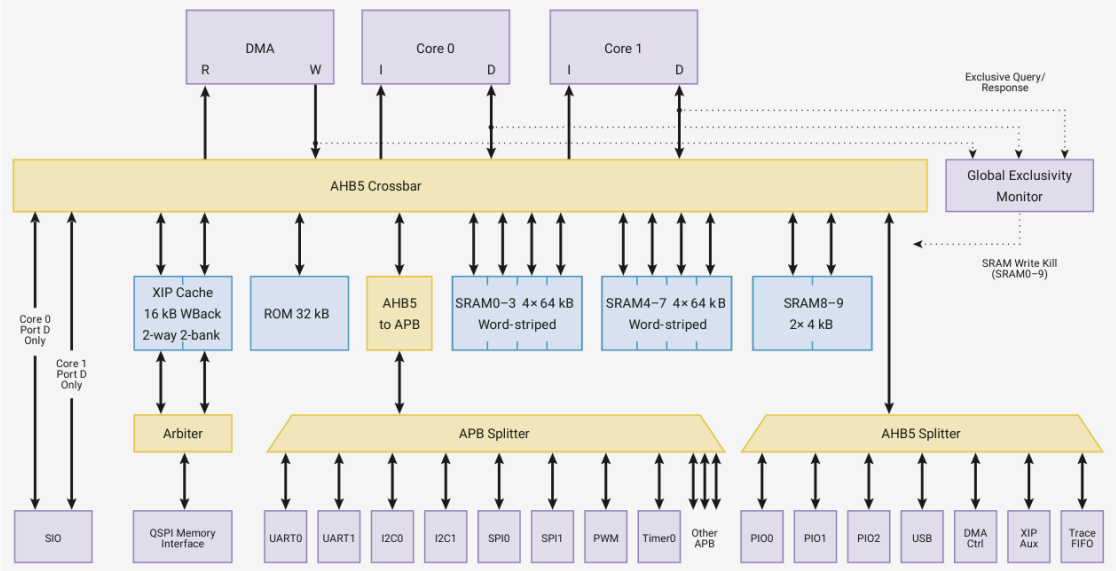
DEBUG DETAIL

sbrk(0) misleads: the SDK hands the whole heap to malloc at the first printf. Free RAM is read with *mallinfo().fordblks*; **__bss_end__** is checked against 0x20080000 after every change.

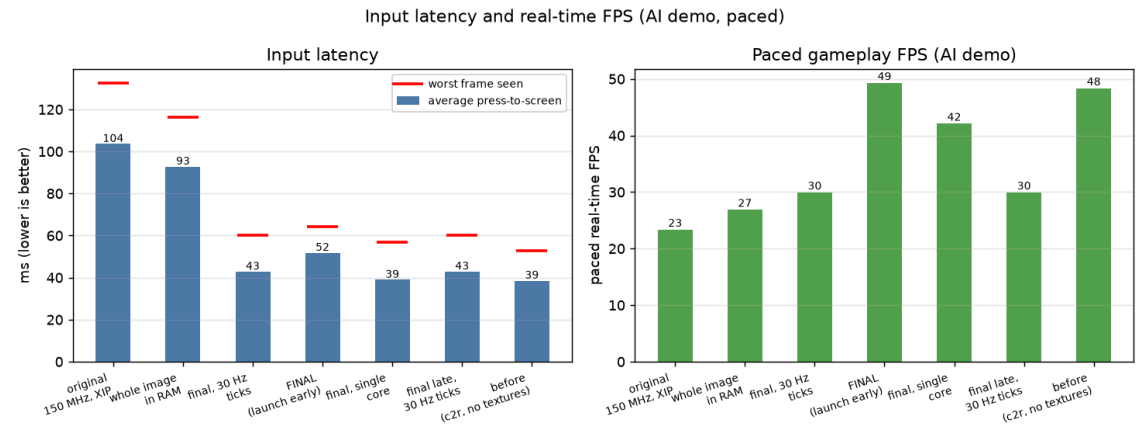
SOFTWARE

Two cores: simulation on core 1, started late

Figure 5.
RP2350 bus fabric overview.



RP2350 datasheet, Fig. 5, bus fabric (global exclusive monitor)



Latency and paced FPS (docs/perf/07)

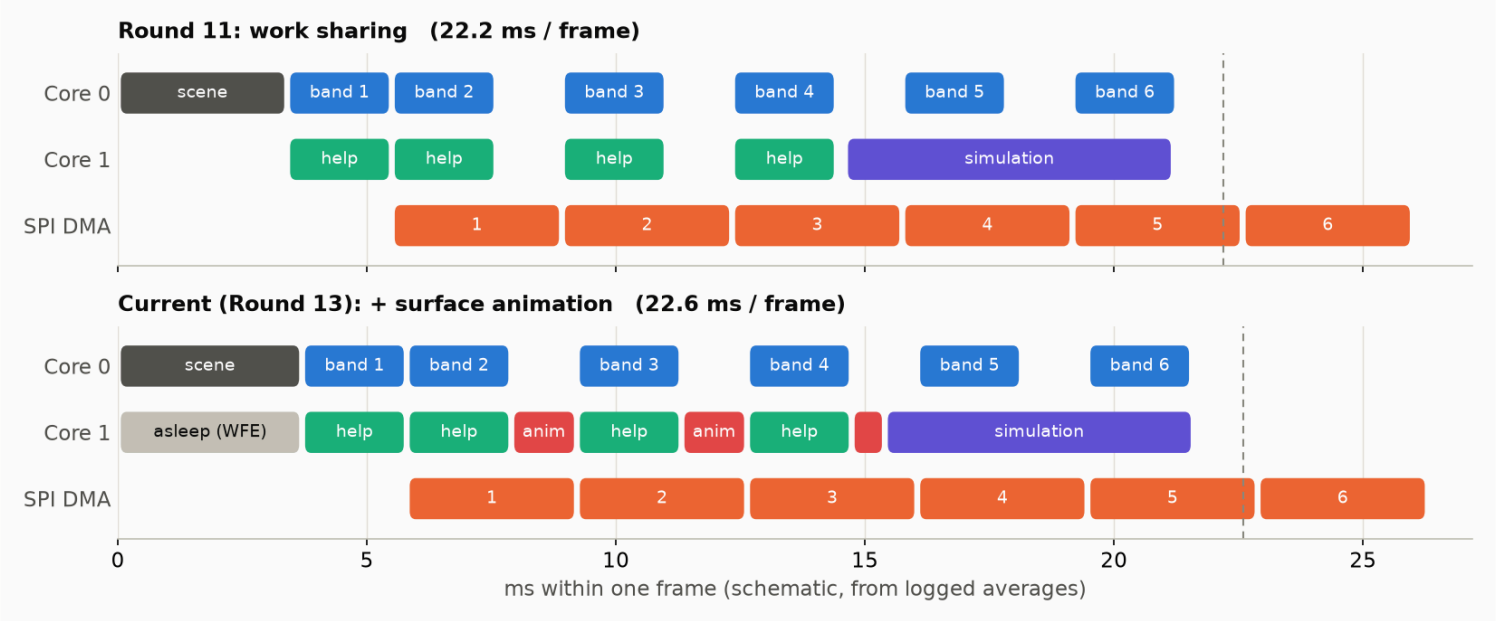
- Hand-off: one atomic state word
- Batch starts when bands left ≈ sim time
- Press-to-screen 104 → 34-39 ms

DEBUG DETAIL

Latency is measured on the board: input sample → last band to DMA + its transfer. The stick is sampled right at launch (*buttons_read(true)*).

SOFTWARE

Work-sharing scheduler (current)



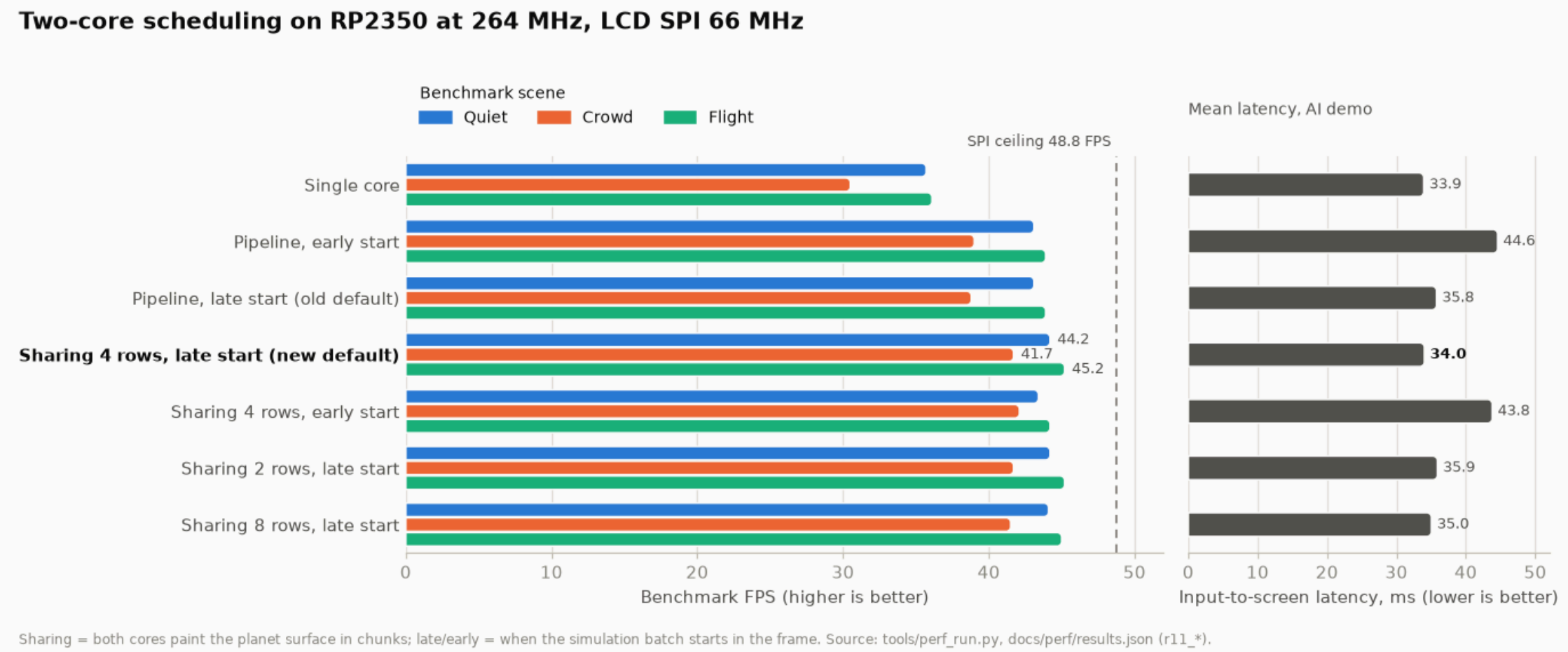
- Planet rows cut into 4-row chunks
- Both cores claim chunks, sim first
- Core 1 paints ~37 % of the planet
- Now SPI-bound: 22.6 vs 20.5 ms

DEBUG DETAIL

Claim word = (job serial « 8) | chunk; a CAS only succeeds for the current job, index 255 closes it before the next band is written. Safe across cores per datasheet §2.1.6 (all SRAM).

SOFTWARE

Seven schedulers measured on the board

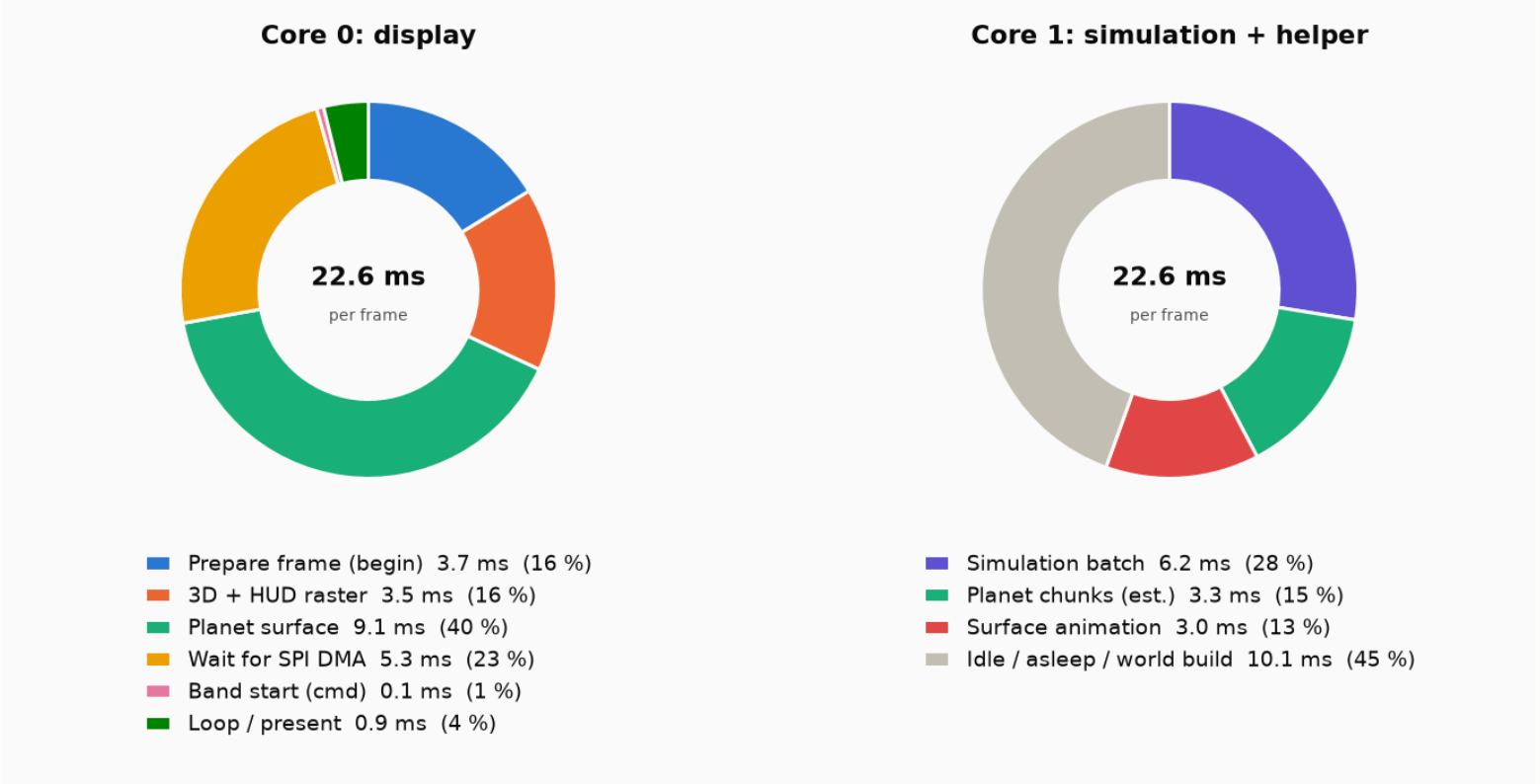


docs/perf/23, tools/perf_run.py, hands-free, 90 s per variant

- Work sharing + late start: **+7.5 %** FPS in the crowd scene, lowest latency of the dual-core variants

SOFTWARE

Time utilisation: where each frame goes



- Core 0 busy **77 %**
- Core 0 waits on SPI: 23 %
- Planet pass: 40 % of core 0
- Core 1 busy **55 %**
- Sim 28 %, planet 15 %, anim 13 %
- Rest: asleep in beginFrame, or idle

docs/perf/25, P + PLN lines of the current build (r13_ship), AI demo

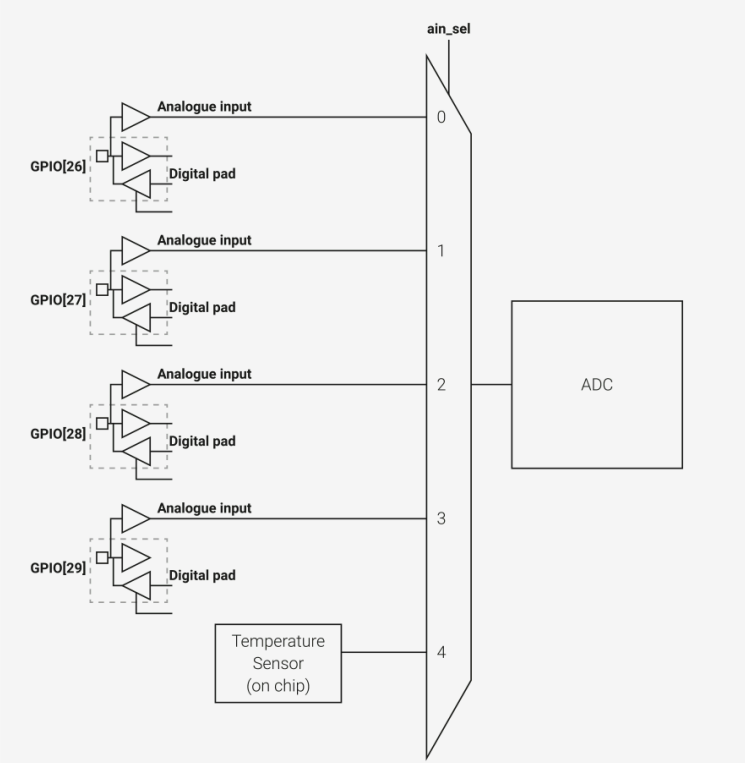
DEBUG DETAIL

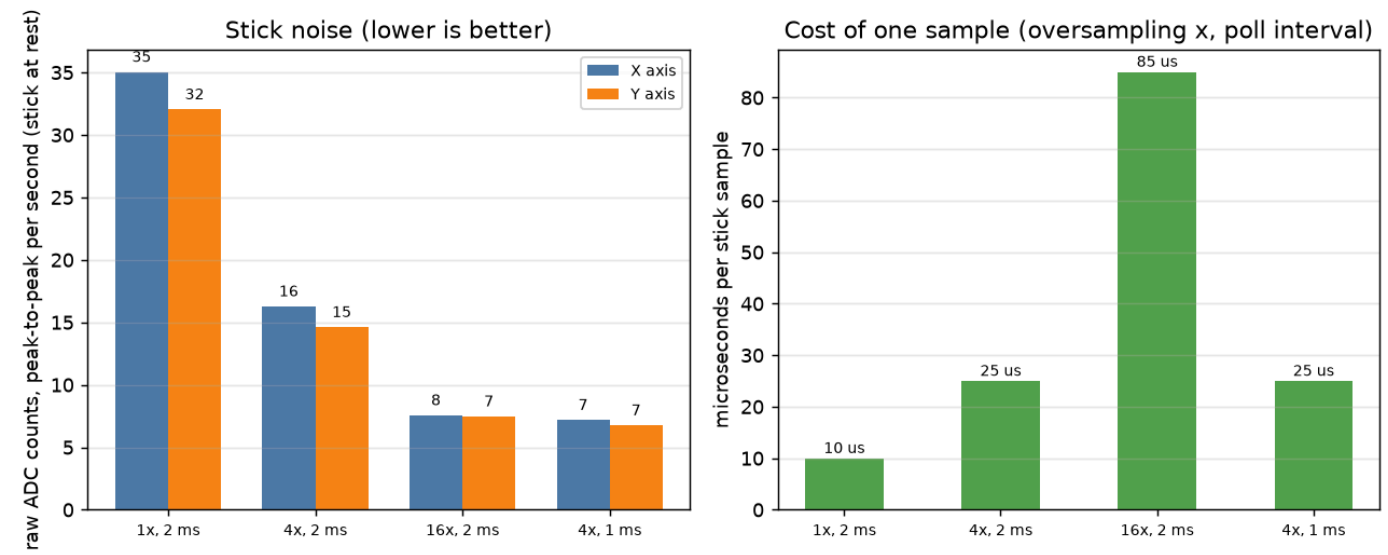
Every *make log* prints this once a second: the **P** line gives per-frame averages of **begin raster wait cmd** on core 0, **sim** on core 1 and the frame period **per**; the **PLN** line gives the planet time inside raster, core 1's **help** % and its surface animation time **anim**. Core 1's planet slice is paint × help %, an estimate. Chart: *docs/perf/time_util_chart.py [log] [phase]*.

HARDWARE + SOFTWARE

Analog stick

Figure 111.
ADC Connection Diagram for QFN-60. This package features four external ADC inputs (0 throu The internal temperature sensor connects to a fifth channel (channel 4). This is functionally th although the underlying hardware is different, to support the additional channels on QFN-80.





Stick noise vs oversampling (docs/perf/08)

- 4× oversampling, 1 ms rate limit
- Deadzone + full scale, –127..127
- Noise 32-35 → 7-16 counts

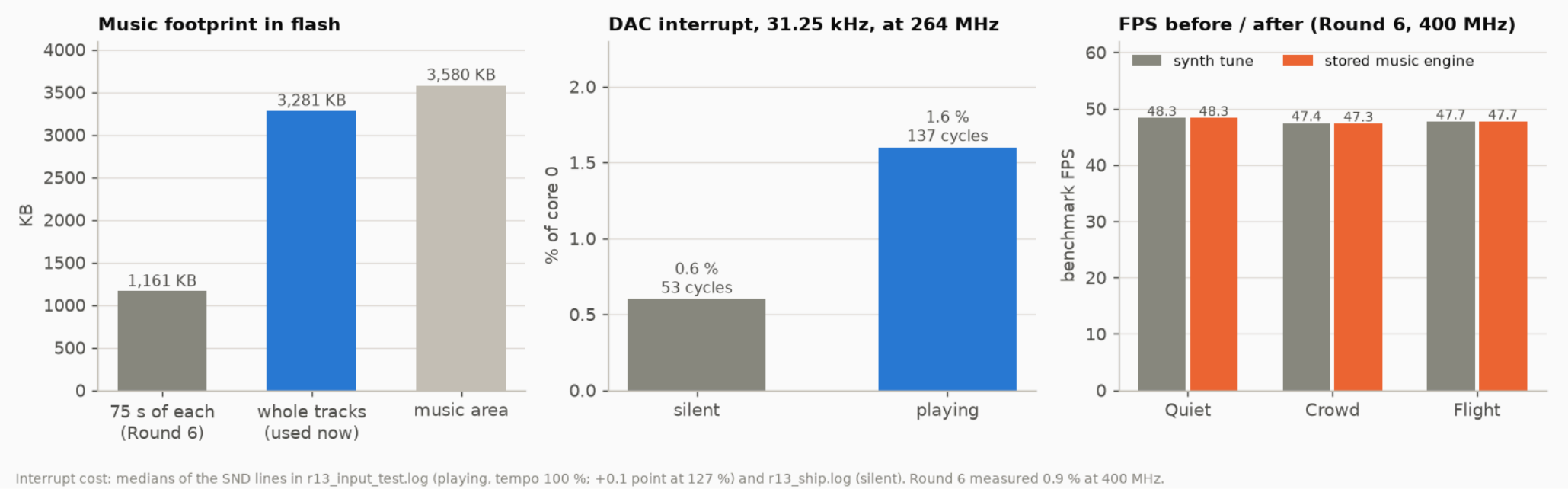
DEBUG DETAIL

The log line carries raw ADC, peak-to-peak and µs per sample (**joy jpp jus**): 16× oversampling cost 3.4× the time (85 vs 25 µs) for no gain, so 4× stays.

RP2350 datasheet, Fig. 111, ADC inputs QFN-60

HARDWARE + SOFTWARE

Audio: PWM DAC + stored music



Interrupt cost: medians of the SND lines in r13_input_test.log (playing, tempo 100 %; +0.1 point at 127 %) and r13_ship.log (silent). Round 6 measured 0.9 % at 400 MHz.

Music footprint, interrupt cost at 264 MHz (docs/perf/15)

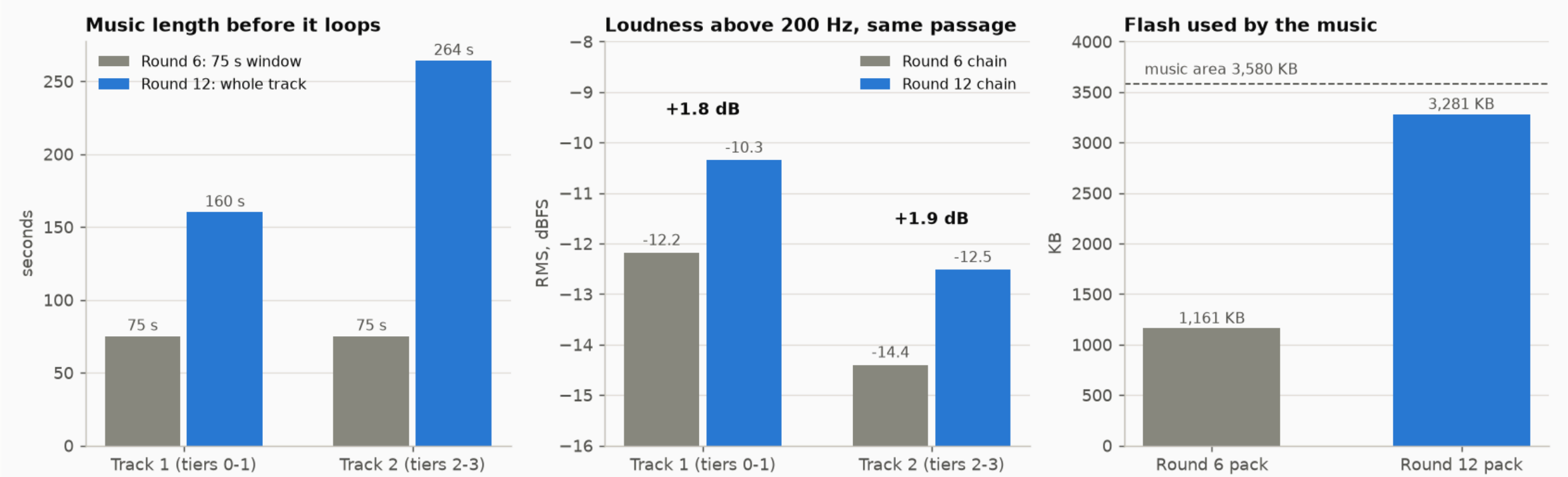
- GP15 → 1 kΩ → NPN → 8 Ω speaker
- 31.25 kHz 8-bit PWM DAC interrupt
- 4-bit IMA ADPCM, 15.6 kHz, from flash
- 1.6 % of core 0 while playing
- Tempo follows the sphere level
- Now: one synthwave track, 134 s (R16)

DEBUG DETAIL

Decoder self-test at boot: the first second of track 1 is decoded on the board and its checksum compared with the packer's (**selftest 1** = bit-exact). Loudness measured in the interrupt on the real output (**music-rms 40/127**), and a DWT cycle counter gives the interrupt's cost (**137 cycles**).

SOFTWARE (ROUND 12)

Music: whole tracks, louder where the speaker plays



Packer: 4-pole high-pass 180 Hz + 3 dB at 2 kHz + 8 dB into the limiter (was 2-pole 90 Hz, flat, 5 dB). Peaks unchanged (0 dBFS). Loudness from the packs decoded on the PC (tools/music_levels.py); on the board: SND music-rms.

docs/perf/26, packs decoded on the PC (tools/music_levels.py)

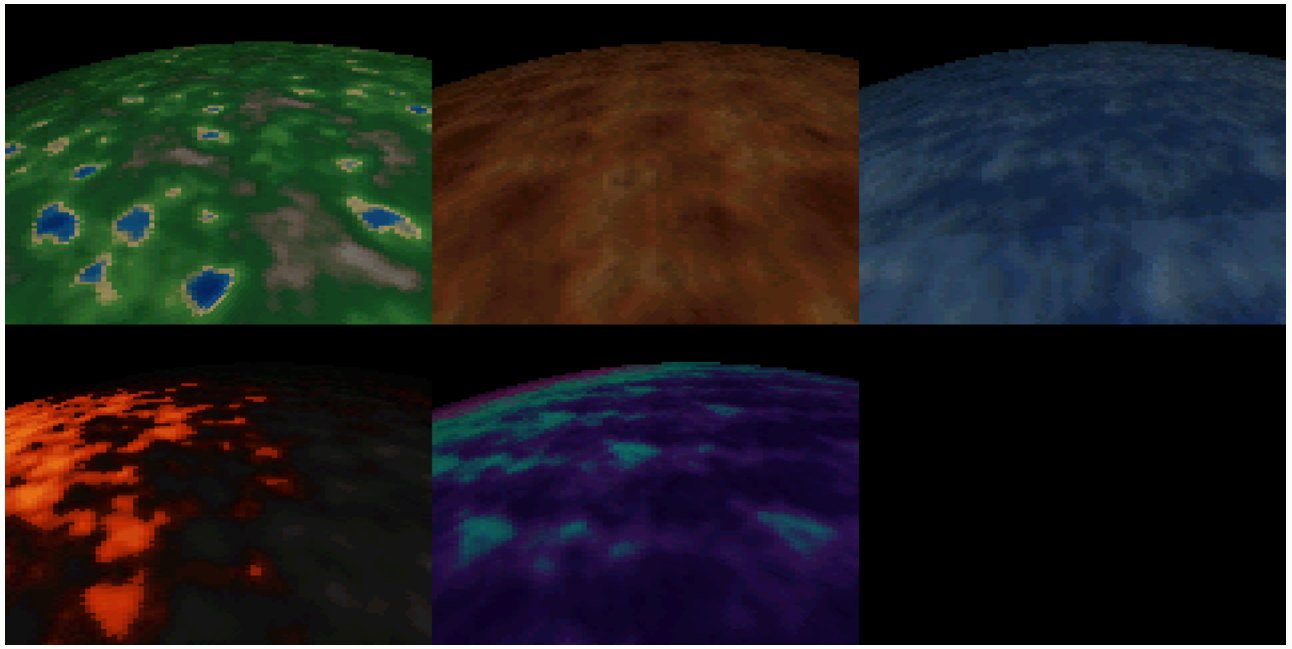
- Whole tracks: 75 s → 160 s and 264 s
- 3,281 KB of the 3,580 KB music area
- R16: synthwave only, 1,034 KB; 3.5 kHz 4-pole low-pass, no 2 kHz lift (highs distorted)

DEBUG DETAIL

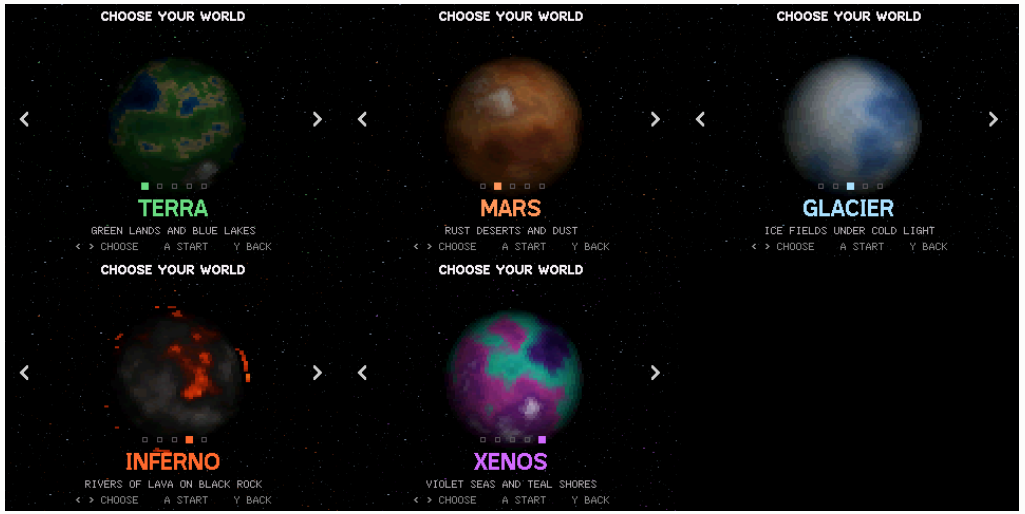
The tracks already hit the limiter: +7 dB of drive gave only +0.7 dB RMS. The headroom was in bass the 8 Ω speaker cannot play: a 4-pole high-pass at 180 Hz and +3 dB at 2 kHz free the limiter for the audible band. On the board: **SND ... music-rms**.

SOFTWARE

World surface



PC renders of the board shader with logged cameras



World select

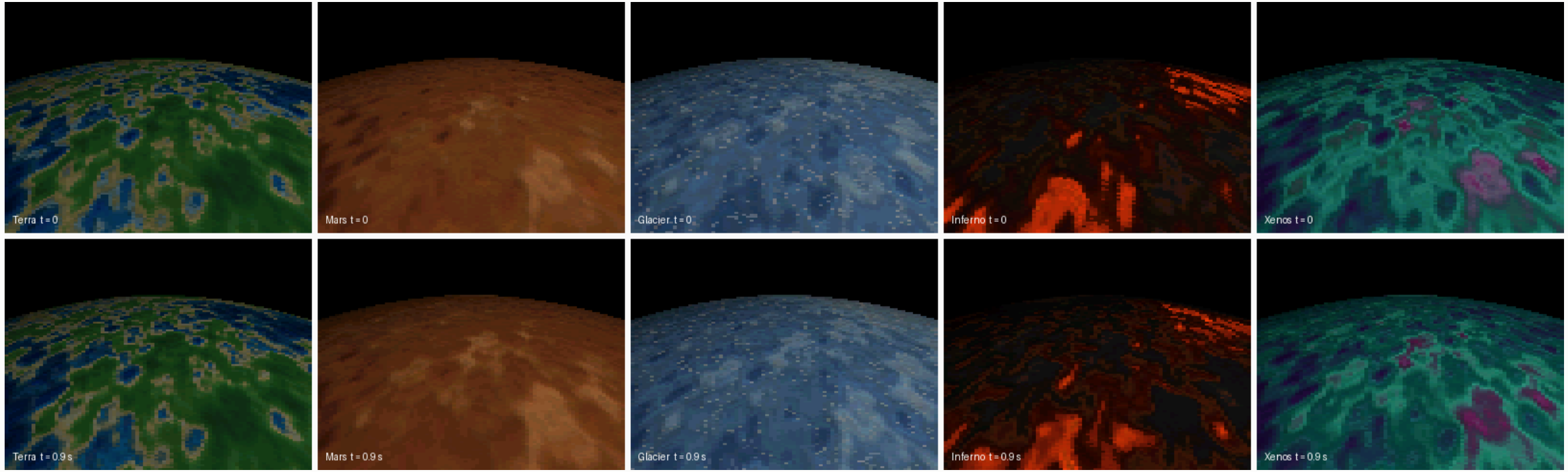
- Ray-sphere per 2×2 block
- Cube height map + hills + grain
- Next world built on core 1 in slices

DEBUG DETAIL

Same C++ shader builds on the PC (*planet_preview*), fed with the board's **CAM** log lines: every look change is checked before flashing. Background build removed a 106 ms frame per world change.

SOFTWARE

Animated surface from core 1's idle time



PC renders of the board code (tools/anim_preview.cpp), two moments 0.9 s apart

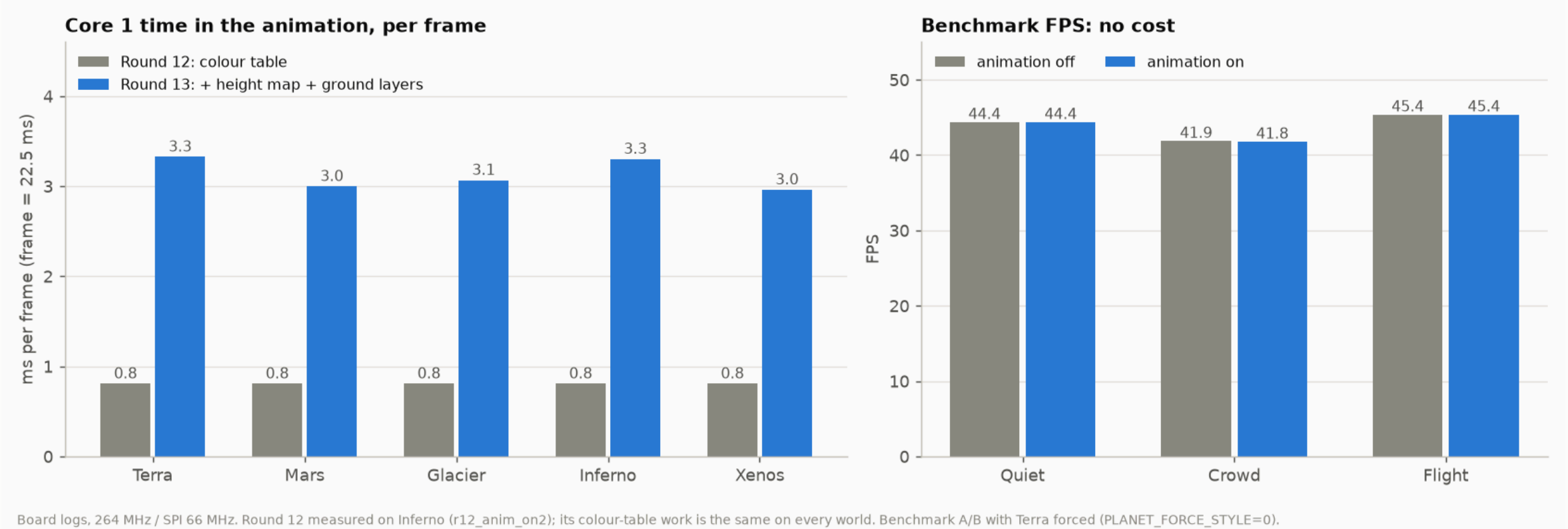
- Height map: waves, beach wash, creeping lava
- Ground tiles: wind, dunes, snow, flowing lumps
- Colour table: surf, glints, glow, embers

DEBUG DETAIL

All integer, 50 passes/s, in units of a few µs. The still copies live in the idle back world set: no new buffer. Core 1 yields between units to a sim batch, an open chunk job and core 0's *beginFrame*. Log: **PLN ... anim n us/frame**.

SOFTWARE

Four times the animation, no FPS cost



Board logs, 264 MHz / SPI 66 MHz. Round 12 measured on Inferno (r12_anim_on2); its colour-table work is the same on every world. Benchmark A/B with Terra forced (PLANET_FORCE_STYLE=0).

docs/perf/27, board logs r12_anim_on2, r13_anim_final, r13_ship, r13_terra_on3/off3

- Core 1: 0.8 → 3.0-3.3 ms per frame
- Benchmark off 44.4/41.9/45.4, on 44.4/41.8/45.4
- Core 1 busy 41 % (R11) → 55 % of a frame

DEBUG DETAIL

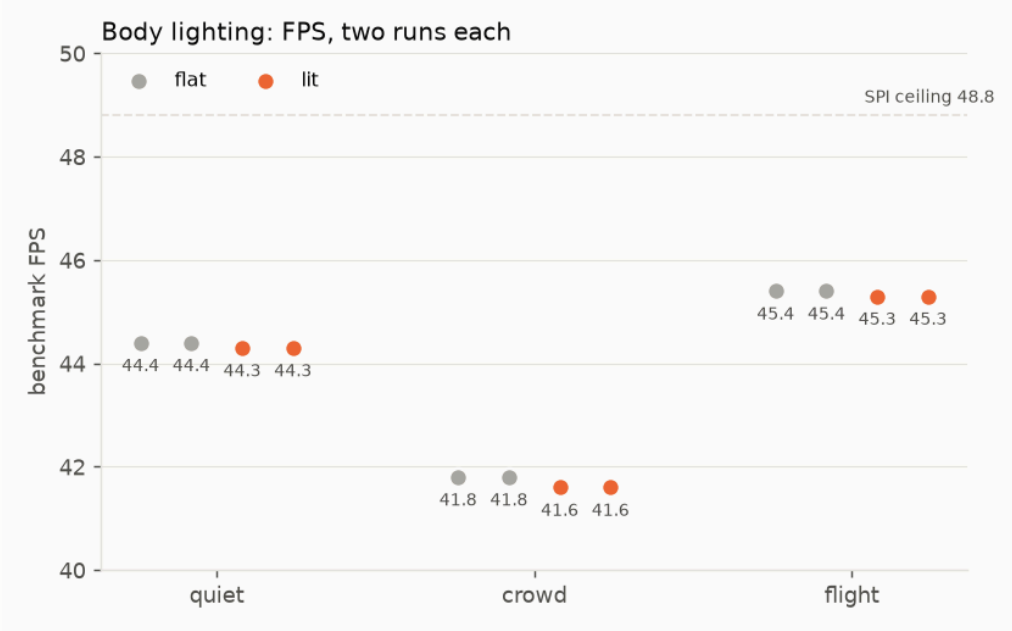
First try, long slices: 0.8 FPS lower in the AI demo. Now the work is cut into µs units with a yield check. Core 1 sleeps in **WFE** during *beginFrame* (core 0 **SEV**s): +0.1 FPS in the benchmark with or without the animation. The demo varies ±0.5 FPS run to run, so the fixed benchmark decides.

SOFTWARE

Bodies lit by the world's sun (Round 17)



PC render of the board code (tools/light_preview.cpp), same frame



Benchmark on the board, docs/perf/30

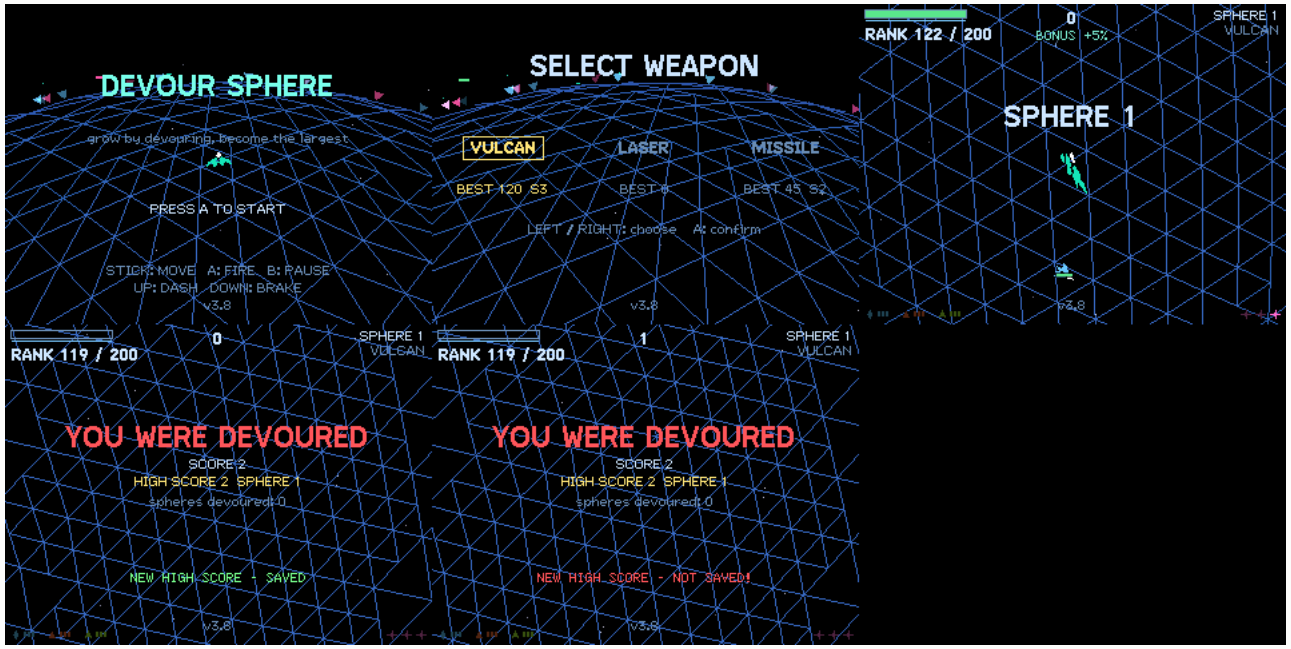
- ShapoGFX ambient + parallel light
- Same sun vector as the ground
- –0.1 to –0.2 FPS, same triangle count

DEBUG DETAIL

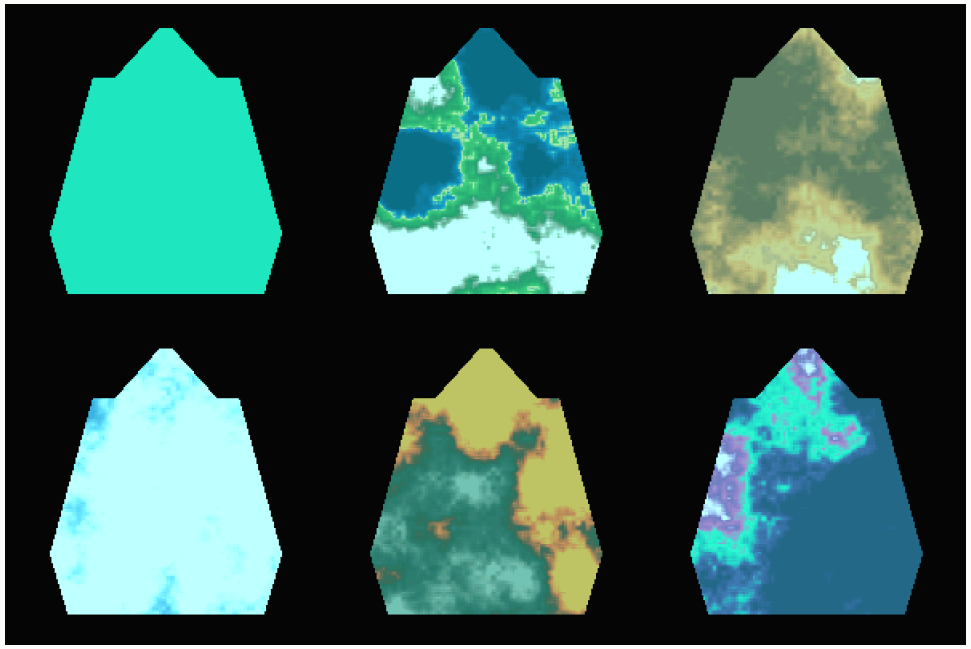
Kites carried a dummy normal (0, 1, 0). Now cross(dir, perp), turned to the eye (double-sided material). One normal per kite keeps it a flat 60 B record. Fork patch 10; levels 45 % + 80 %.

SOFTWARE

Screens and player skin



Title, weapon select, play, game over (PC renders)



Player skinned with the world's terrain



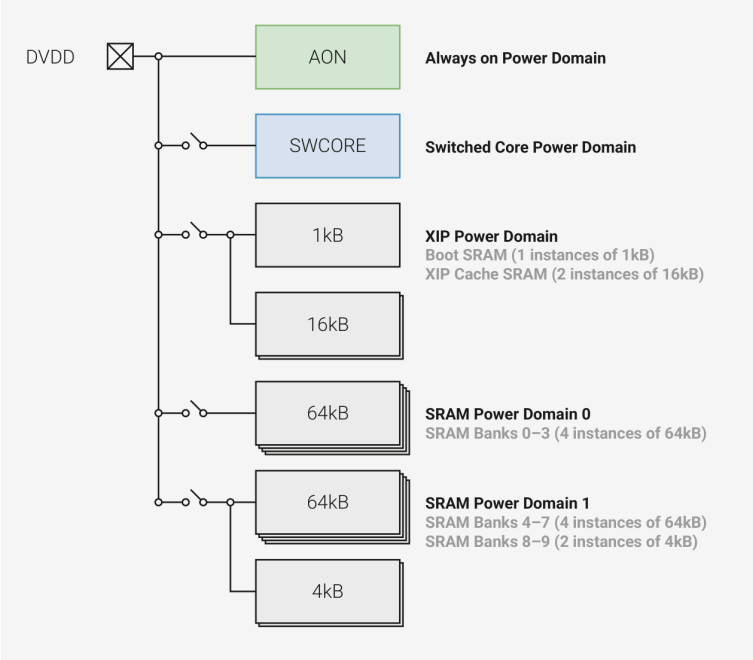
Cut scene storyboard

- Renderer patched via small hooks
- Star field sized from free heap
- UI uses the console's names: A, B

HARDWARE + SOFTWARE

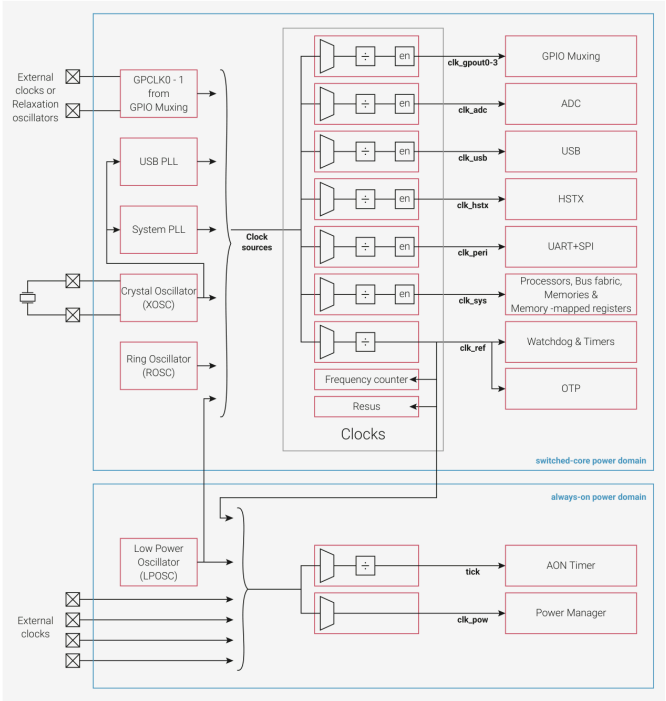
Sleep: hold B, wake on B at the same frame

Figure 21.
Core power domains



RP2350 datasheet, Fig. 21, core power domains

Figure 36.
Clocks overview



RP2350 datasheet, Fig. 36, clocks

- Screen + speaker off
- Both cores in deep sleep (WFI/WFE)
- 64 MHz via PLL, then 1.10 V
- Unused peripheral clocks gated
- B = GPIO IRQ; 1 s alarm feeds watchdog

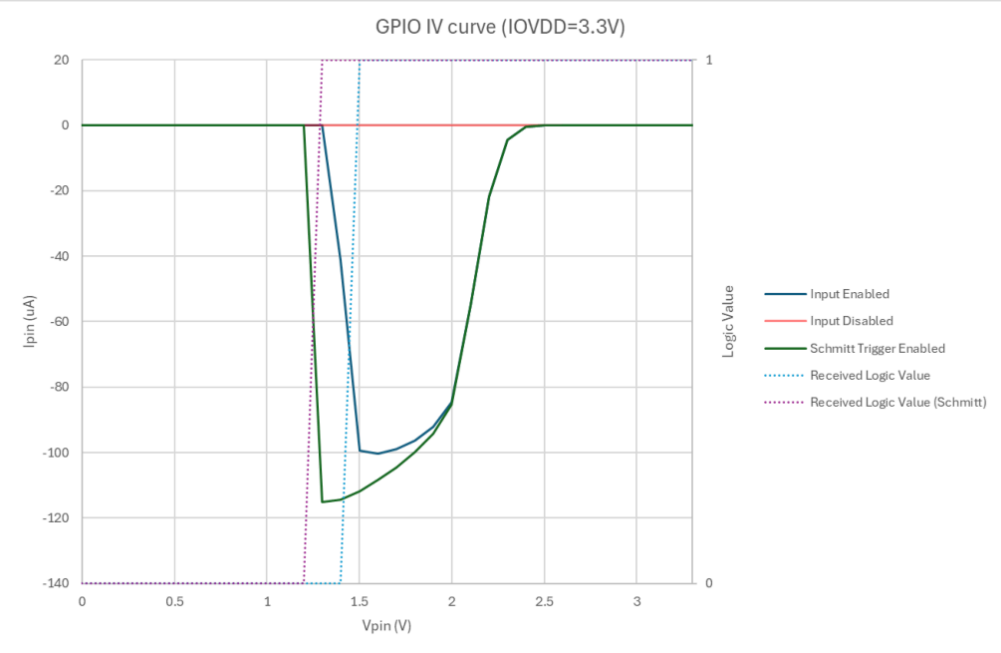
DEBUG DETAIL

Order matters: clock down → voltage down; on wake voltage up, 1 ms, clock up, then sound and LCD. Log lines **SLP wake 1/2/3** and **WAKE** at the same tick prove each step; USB stays up through the sleep.

DATASHEET ERRATA

RP2350-E9 and E12 applied to the sleep

Figure 157.
GPIO Pad leakage for IOVDD=3.3 V



RP2350 datasheet, Fig. 157, pad leakage (E9, chip A2)

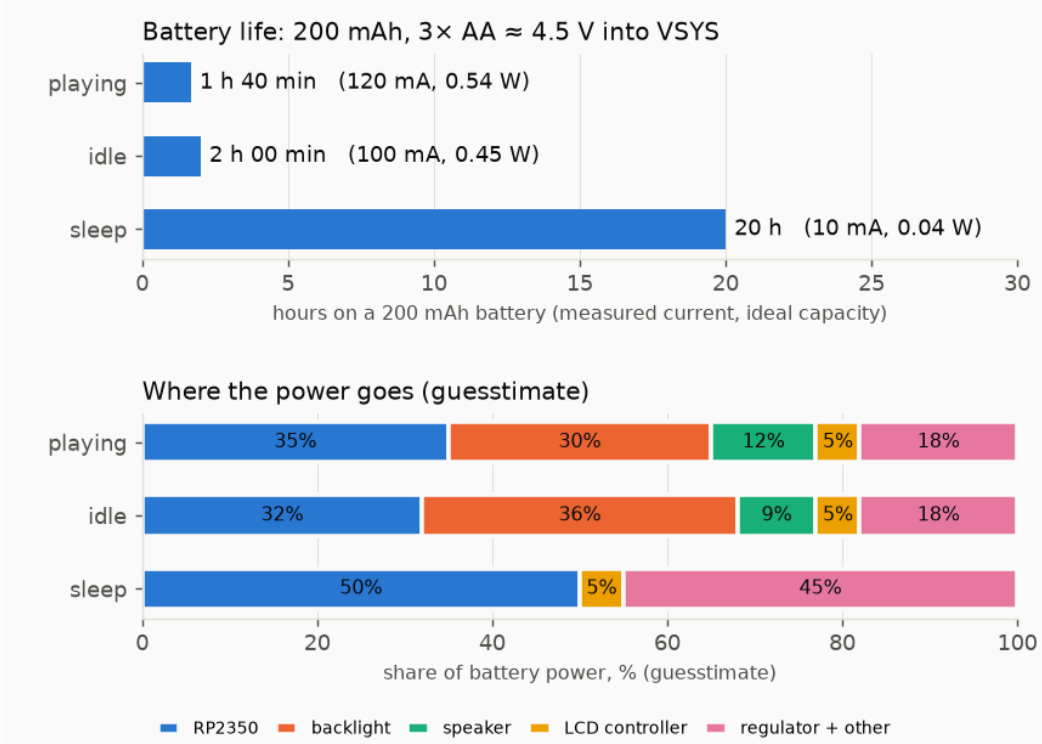
- E9: input pad between VIL/VIH leaks ~120 µA
- Speaker pin: driven low, input off
- All other pads: input off, pulls off
- E12: clk_sys ≥ 1.1 × clk_usb → 64 MHz
- Die temp falls in sleep: 60.2 → 57.1 °C

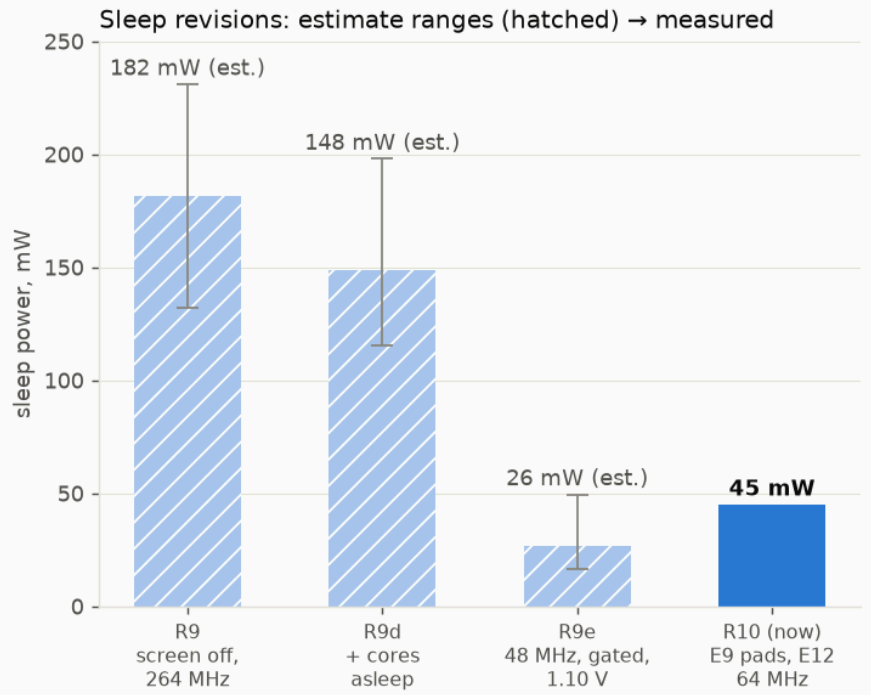
DEBUG DETAIL

The log prints the stepping (**chip A2**) and how many pads changed (**pads IE-off 9**). A *static_assert* pins the E12 rule. Every datasheet-derived change is listed in *docs/datasheet/IMPLEMENTATION.md*.

HARDWARE + SOFTWARE

Battery current, measured





User's meter, 3× AA (≈ 4.5 V). Life = 200 mAh / current. Split and hatched bars are estimates

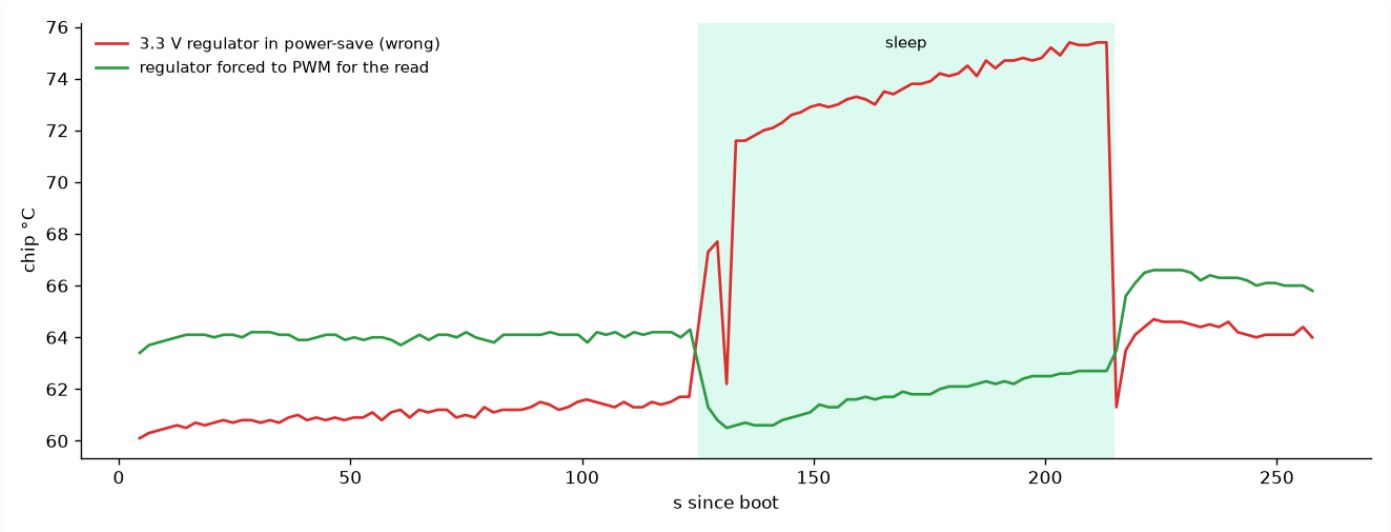
- 200 mAh: **1 h 40 min** playing, **20 h** asleep
- Sleep ≈ 1/12 of play: backlight + speaker off
- Measured values sit inside the estimate ranges

DEBUG DETAIL

Sleep path: R9 screen + speaker off → R9d cores in WFI/WFE → R9e 48 MHz via PLL, peripheral clocks gated, 1.10 V → R10 E9 pads input-off, 64 MHz for E12. The old board's 380/200 mA was a board fault (> 2 A in BOOTSEL).

HARDWARE

Measuring die temperature correctly



- Sensor: ADC ch. 4, 0.706 V @ 27 °C
- –1.721 mV/°C (datasheet §12.4.6)
- GPIO23 high: regulator in PWM mode
- Up to +11 °C error removed

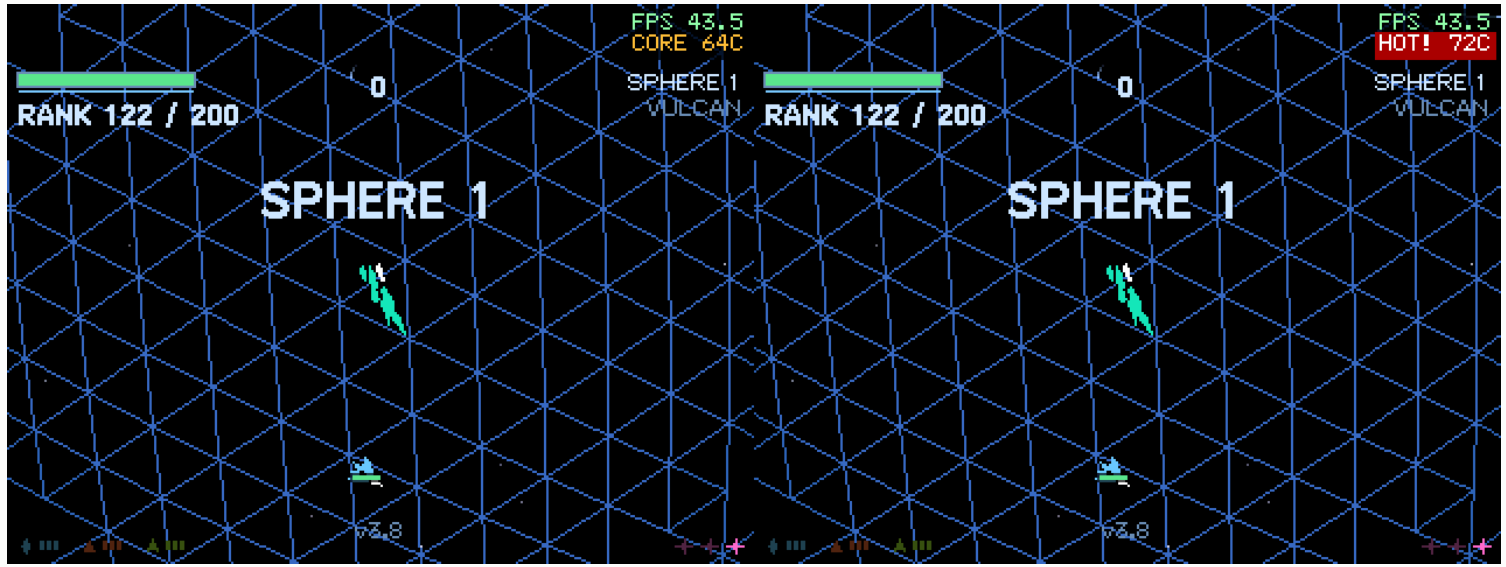
Same run read both ways (docs/perf/22)

DEBUG DETAIL

At low load the Pico 2's 3.3 V regulator switches to power-save mode; its ripple moved the ADC reference. Holding GPIO23 high for 2 ms around each read gave steady values (60.5-62.7 °C asleep).

SOFTWARE

Core temperature under the FPS



PC render: WARM (amber) and HOT (red, blinking)

< 60 °C	CORE nnC, green
≥ 60 °C	WARM, amber
≥ 70 °C	HOT!, red, blinking
125 °C	junction max (Table 1680)
85 °C	ambient max (Table 1682)

DEBUG DETAIL

Non-blocking: frame N sets GPIO23, frame N+1 (≥ 2 ms later) waits 50 µs and reads 20 conversions (~0.1 ms). FPS unchanged. Levels leave 2 °C below their threshold; **TEMP level** log lines record each change.

HARDWARE LIMITS

Board and chip constraints, and the answer to each

LIMIT	EFFECT	ANSWER
RP2350 A2	E9 pad leakage	inputs off on idle pads in sleep
12 MHz crystal	PLL steps only (350 MHz not reachable)	264 MHz / 4 = 66 MHz SPI
520 KB SRAM, no PSRAM	no framebuffer, no full copy-to-RAM	bands, 3 libraries in RAM
4 MB flash	music competes with code	134 s synthwave in 1,034 KB of 3,580 KB
2 buttons used	GP10 floats, GP11 tied low (first board)	all UI on A + B + stick
Analog stick	12-bit ADC noise at rest	4× oversampling, deadzone
Speaker	one PWM pin, NPN, 8 Ω	31.25 kHz 8-bit DAC in one ISR
3.3 V regulator	power-save ripple shifts the ADC	GPIO23 high during temp reads
GPIO24	no usable VBUS sense	USB clocks kept in sleep
No current sensor	firmware cannot read power	user's meter at the battery; die temperature

DEBUG

The toolbox behind every number

USB log	P-line: FPS, sim/begin/raster/wait µs, latency, state, button mask, stick
PLN	planet paint µs, core 1 help %, surface animation µs, world style
CFG	real clocks, SPI rate, fallback, breadcrumbs, scheduler mode
perf_run.py	build → flash → log → results.json, hands-free, 90 s per variant
Benchmark	3 fixed scenes + paced AI demo inside the firmware
Scripted tests	GAME_INPUT_TEST (pause, restart, deaths), GAME_TEMP_TEST
FAULT	HardFault PC/LR/CFSR kept in .noinit RAM, printed after reset
PC previews	planet, HUD, intro, skin: the board's own code on the PC
Datasheet	tools/ds_text.py + IMPLEMENTATION.md: section → code → status

CURRENT REVISION

Where it stands

42-45
FPS (benchmark 44.3 / 41.6 / 45.3)

38 ms
input to screen (AI demo)

264 MHz
@ 1.15 V, SPI 66 MHz

36 %
of the planet painted by core 1

278 KB
firmware; 134 s of music at 512 KB

120 mA
playing on 3× AA; 10 mA asleep

On the board with this build (log, Round 17): benchmark and AI-demo frame rate, latency, planet sharing, music found and decoded bit-exact. Measured by the user: battery current (idle 100, play 120, sleep 10 mA). Checked by the user on the LCD: ST7789 colours and orientation, the synthwave music. Not yet checked: the lit bodies on the LCD, the joystick up/down fix in play, and sleep/wake on the new board.

CREDITS

Thank you

Devour Sphere on a Pico 2, built on the work of others

GRAPHICS

ShapoGFX by Shapoco

2D/3D software renderer for microcontrollers: the scanline 3D scene into band buffers, the HUD, fonts and screens.
MIT License · github.com/shapoco/shapo-gfx

SDK	Raspberry Pi pico-sdk and picotool (BSD-3-Clause)
Chip	RP2350 datasheet (Raspberry Pi Ltd), quoted figures and tables
Music	synthwave track, packed as 4-bit ADPCM from flash